

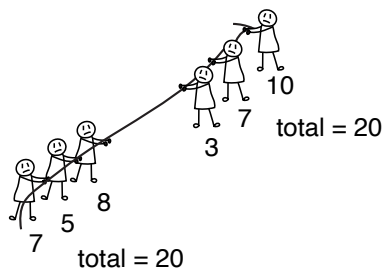
Cabo de guerra

Autor: Mateus Bezrutchka

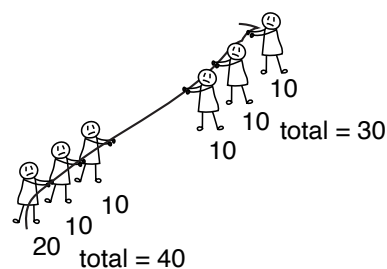
Nome do arquivo: `cabo.c`, `cabo.cpp`, `cabo.java`, `cabo.js` ou `cabo.py`

Na mais nova conferência da OBI (*Organização de Brincadeiras Infantis*), os participantes estão explorando brincadeiras de competição em times, como o clássico jogo do *Cabo de Guerra*. A versão de *Cabo de Guerra* jogada na OBI é sempre disputada entre dois times, cada um com três integrantes. Cada time se reúne em uma das pontas de uma corda e então os times começam a puxar a corda em direções opostas. O time vencedor é aquele que consegue puxar a corda em sua direção, fazendo com que o time oponente a solte.

No banco de dados da OBI, cada jogador possui uma *força*, representada por um número inteiro entre 1 e 100, e a *força total* de um time é definida como a soma das forças de seus integrantes. Por exemplo, um time cujos integrantes têm forças 7, 5 e 8 tem força total $7 + 5 + 8 = 20$. Os estudos da OBI concluíram que, se os dois times tiverem forças totais diferentes, a partida de *Cabo de Guerra* sempre acaba em menos de 5 minutos. Porém, se os dois times tiverem exatamente a mesma força total, o jogo entra em um *impasse* e os participantes continuam puxando a corda por 12 horas!



Impasse



Sem Impasse

A coordenação da OBI está organizando partidas de *Cabo de Guerra*, mas precisa evitar qualquer impasse. Por isso, eles pediram a sua ajuda: dadas as forças dos seis participantes de uma partida, determine se existe alguma chance de impasse – ou seja, se existe um modo de dividir os seis participantes em dois times tal que cada participante pertença a exatamente um dos dois times, cada time tenha três integrantes, e os dois times tenham a mesma força total.

Entrada

A única linha da entrada contém seis números inteiros positivos X_i ($1 \leq i \leq 6$), indicando as forças dos seis participantes.

Saída

Seu programa deverá imprimir uma única linha contendo um único caractere: ‘S’ (sem aspas), se existe um modo de dividir os times que causa impasse, ou ‘N’ (sem aspas), se não existe modo de causar impasse.

Restrições

- $1 \leq X_i \leq 100$ para $1 \leq i \leq 6$

Informações sobre a pontuação

A tarefa vale 100 pontos.

Exemplos

Exemplo de entrada 1 7 5 3 8 10 7	Exemplo de saída 1 S
---	--------------------------------

Explicação do exemplo 1: Esse exemplo é representado pela figura (a). Podemos montar um dos times com os participantes de forças 7, 5 e 8 e o outro time com os participantes de forças 3, 10 e 7. Desse modo, ambos os times têm força total 20.

Exemplo de entrada 2 10 10 10 20 10 10	Exemplo de saída 2 N
--	--------------------------------

Explicação do exemplo 2: Esse exemplo é representado pela figura (b). Em qualquer divisão válida dos participantes em dois times, um dos times terá força total $20 + 10 + 10 = 40$ e o outro terá força total $10 + 10 + 10 = 30$, logo as forças totais não podem ser iguais.

Exemplo de entrada 3 5 5 6 6 8 8	Exemplo de saída 3 S
--	--------------------------------

Exemplo de entrada 4 4 8 12 24 20 16	Exemplo de saída 4 N
--	--------------------------------

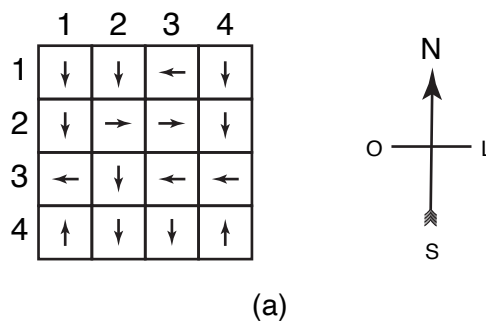
Tesouro

Autor: Mateus Bezrutchka

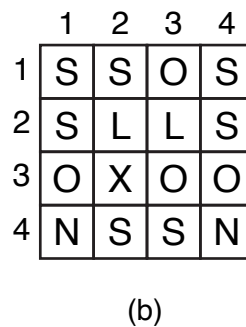
Nome do arquivo: `tesouro.c`, `tesouro.cpp`, `tesouro.java`, `tesouro.js` ou `tesouro.py`

Eduardo é um estudante do departamento de Arqueologia da Unicamp que recentemente encontrou um mapa da Quadradônia, uma ilha histórica cujos habitantes adoravam quadrados. A ilha pode ser representada como um quadrado de lado M dividido em M^2 quadrados unitários com coordenadas de $(1, 1)$ a (M, M) , com a primeira coordenada indicando a linha (numerada de 1 a M de Norte a Sul) e a segunda indicando a coluna (numerada de 1 a M de Oeste a Leste). Cada um desses M^2 quadrados representa uma *região* da ilha.

A Quadradônia está atualmente inabitada: seus habitantes fugiram quando a areia que cobre a ilha se tornou movediça! A areia de cada uma das regiões se move em um sentido específico, que pode ser para o Norte, Sul, Leste ou Oeste. Uma pessoa que pisa em uma das regiões da ilha é imediatamente arrastada pela areia, sendo movida da sua região atual para uma das regiões vizinhas (de acordo com o sentido de movimento da areia na região atual), com a velocidade de uma região por minuto. A figura abaixo ilustra uma representação da Quadradônia para $M = 4$, com o sentido de movimento da areia indicado em cada região.



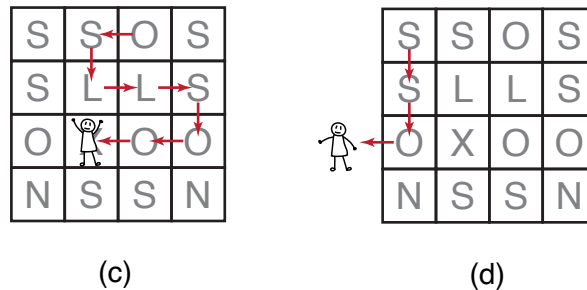
O mapa encontrado por Eduardo indica a posição de um tesouro escondido em uma das regiões da Quadradônia, que está marcada com um X. Para todas as outras regiões, o mapa indica, com um caractere, o sentido no qual a areia se move na região (N para Norte, S para Sul, L para Leste, O para Oeste). A figura (b) indica o mapa do tesouro correspondente à figura (a), assumindo que o tesouro está escondido na região $(3, 2)$.



Interessado no tesouro, Eduardo escolheu uma das regiões da ilha para sobrevoar e saltar de paraquedas, pousando na região escolhida. Ele percebeu que a escolha dessa região inicial é muito importante: dependendo da região inicial, ele pode demorar mais ou menos tempo para chegar à

região que contém o tesouro. Além disso, é possível que ele eventualmente seja arrastado para fora das bordas da ilha, caindo no mar, ou ainda que seja arrastado na areia movediça para sempre, sem nunca chegar ao tesouro.

Considere os seguintes percursos possíveis:



A figura (c) indica o percurso de Eduardo se ele saltar na região (1,3); nesse caso, ele chega ao tesouro em 7 minutos. A figura (d) indica o percurso para região inicial (1,1); nesse caso, ele é jogado ao mar após chegar à região (3,1).

Eduardo pediu sua ajuda: dado o mapa do tesouro da Quadradônia e as coordenadas da região inicial dele (ou seja, a região na qual ele vai pousar), determine o tempo em minutos que ele levará para chegar à região que contém o tesouro. Caso Eduardo nunca chegue ao tesouro, indique se ele cairá no mar ou ficará preso na areia movediça para sempre.

Entrada

A primeira linha da entrada é composta por um inteiro M , indicando que a Quadradônia é composta por $M \times M$ regiões.

As próximas M linhas descrevem o mapa do tesouro e possuem M caracteres cada. R_{ij} , o j -ésimo caractere da i -ésima dessas linhas, representa a região (i, j) e pode ser:

- ‘X’, se a região (i, j) contém o tesouro;
- ‘N’, ‘S’, ‘L’ ou ‘O’, de acordo com o sentido de movimento da areia na região.

A última linha da entrada contém dois inteiros A e B , as coordenadas da região inicial de Eduardo (ou seja, a região na qual ele vai pousar).

Saída

Seu programa deverá produzir uma única linha contendo um único inteiro:

- Se Eduardo eventualmente chega à região que contém o tesouro, imprima o tempo em minutos que ele leva para chegar ao tesouro.
- Se Eduardo continua sendo arrastado na areia para sempre, sem nunca chegar ao tesouro, imprima 0.
- Se Eduardo sai das bordas da ilha e é jogado ao mar, imprima -1.

Restrições

- $2 \leq M \leq 1\,000$

- $R_{ij} = 'N', 'S', 'L', 'O'$ ou $'X'$ para todos $1 \leq i, j \leq M$
- Existe exatamente um par (i, j) para o qual $R_{ij} = 'X'$
- $1 \leq A, B \leq M$
- $M_{AB} \neq 'X'$ (ou seja, a região inicial e a região do tesouro são distintas)

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima:

- **Subtarefa 1 (30 pontos):** $M = 2$. (Veja os exemplos 3 e 4.)
- **Subtarefa 2 (40 pontos):** $M \leq 50$ e é garantido que Eduardo consegue chegar à região que contém o tesouro.
- **Subtarefa 3 (30 pontos):** Nenhuma restrição adicional.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (*elas não precisam ser resolvidas em ordem*). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
4 SSOS SLLS OXOO NSSN 1 3	7

Explicação do exemplo 1: Esse é o exemplo da figura (c).

Exemplo de entrada 2	Exemplo de saída 2
4 SSOS SLLS OXOO NSSN 1 1	-1

Explicação do exemplo 2: Esse é o exemplo da figura (d).

Exemplo de entrada 3	Exemplo de saída 3
2 SO XO 2 2	1

Exemplo de entrada 4	Exemplo de saída 4
2 L0 X0 1 2	0

Explicação do exemplo 4: Eduardo ficará para sempre sendo arrastado entre as regiões $(1, 2)$, cuja areia se move para o Oeste, e $(1, 1)$, cuja areia se move para o Leste.

Metrônibus

Autores: André Amaral de Sousa e Mateus Bezrutchka

Nome do arquivo: `metronibus.c`, `metronibus.cpp`, `metronibus.java`, `metronibus.js` ou `metronibus.py`

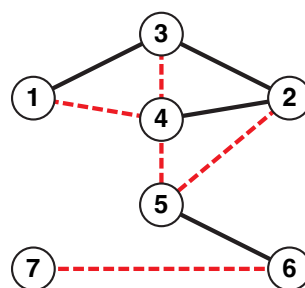
Ênia, a atual rainha da Nlogônia, é fascinada por meios de transporte, especialmente metrô e ônibus. Como consequência, a Nlogônia possui a melhor rede de transporte público do mundo inteiro, a *Metrônibus*.

Existem N estações espalhadas por todo o reino, que são conectadas por dois *sistemas de transporte*: ônibus, que utiliza rodovias, e metrô, que utiliza trilhos. Assim, no Metrônibus existe transporte direto entre duas estações se elas estão ligadas por uma rodovia ou por um trilho. Toda ligação, independente de ser rodovia ou trilho, pode ser utilizada em ambas as direções. Existe no máximo uma ligação entre cada par de estações (ou seja, um par de estações não pode estar ligado por ambos um trilho e uma rodovia).

Para utilizar o Metrônibus, o usuário precisa comprar passagens. Ênia estabeleceu as seguintes regras para estimular o transporte público:

- Toda passagem tem um preço fixo P , independente da estação e do sistema (metrô ou ônibus).
- A passagem só deve ser paga quando o usuário entrar no sistema de transporte.
- O usuário só pode entrar em um sistema por vez (ou seja, para entrar no sistema de metrô ele precisa sair do sistema de ônibus, e vice-versa).
- Após pagar o preço P para entrar em um sistema, o usuário pode se locomover o quanto desejar usando ligações daquele sistema, contanto que ele não saia do sistema. Caso o usuário troque de sistema, ele deverá pagar por uma nova passagem para retornar ao sistema original.

Por exemplo, considere a seguinte rede do Metrônibus:



Existem 7 estações e 9 ligações no total, das quais 4 são trilhos do metrô e as outras 5 são rodovias para ônibus. Suponha que o preço da passagem seja $P = 5$ e que o usuário deseja fazer uma viagem da estação 1 para a estação 7. Temos várias opções de trajeto, como por exemplo (as setas indicam a ordem em que visitamos as estações):

- **Viagem 1:** $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$
 - A ligação entre as estações 1 e 3 usa o metrô, então precisamos entrar no sistema de metrô pela primeira vez, pagando $P = 5$.

- O trecho $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$ é todo percorrido de metrô, logo não precisamos pagar de novo.
 - $4 \rightarrow 5$ usa ônibus, então precisamos pagar $P = 5$ para trocar de metrô para ônibus.
 - $5 \rightarrow 6$ usa o metrô, então pagamos mais $P = 5$ para reentrar no sistema de metrô.
 - $6 \rightarrow 7$ usa ônibus, então precisamos pagar mais $P = 5$ para entrar no sistema de ônibus novamente e chegar à estação 7.
 - No total, a viagem custa $5 + 5 + 5 + 5 = 20$.
- **Viagem 2:** $1 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$
- A ligação entre as estações 1 e 4 usa ônibus, que estamos utilizando pela primeira vez, então é necessário pagar $P = 5$.
 - $4 \rightarrow 5$ também usa ônibus, ou seja, continuamos no mesmo sistema de transporte e não é necessário pagar outra passagem.
 - $5 \rightarrow 6$ usa o metrô, que vamos utilizar pela primeira vez, logo é necessário pagar $P = 5$.
 - $6 \rightarrow 7$ usa o sistema de ônibus, do qual saímos e precisamos pagar $P = 5$ para reentrar.
 - No total, a viagem custa $5 + 5 + 5 = 15$.

Existem outras viagens que começam na estação 1 e terminam na estação 7. Porém, o menor custo total entre todas essas viagens é 15 (correspondente à viagem 2).

Ênia deseja saber o menor custo total para ir da estação A para a estação B , e para isso te contratou. Responda corretamente, ou ela te expulsará da Nlogônia!

Entrada

A primeira linha da entrada contém quatro inteiros N , K_1 , K_2 , e P , representando o número de estações, o número de ligações (trilhos) no sistema de metrô, o número de ligações (rodovias) no sistema de ônibus, e o preço da passagem, respectivamente.

As próximas K_1 linhas contém a descrição das ligações do metrô. A i -ésima dessas linhas terá dois inteiros V_i e U_i , representando que há um trilho entre as estações V_i e U_i .

As próximas K_2 linhas contém a descrição das ligações de ônibus. A j -ésima dessas linhas terá dois inteiros X_j e Y_j , representando que há uma rodovia entre as estações X_j e Y_j .

A última linha da entrada contém dois números inteiros, A e B , representando a estação inicial e a estação final da viagem requisitada pela rainha.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o menor custo total de uma viagem que começa na estação A e termina na estação B .

Se não for possível fazer uma viagem de A a B usando o Metrônibus, imprima -1 .

Restrições

- $2 \leq N \leq 100\,000$
- $1 \leq K_1 + K_2 \leq 100\,000$
- $1 \leq P \leq 1\,000$
- $1 \leq V_i, U_i \leq N$ para todo $1 \leq i \leq K_1$
- $V_i \neq U_i$ para todo $1 \leq i \leq K_1$

- $1 \leq X_j, Y_j \leq N$ para todo $1 \leq j \leq K_2$
- $X_j \neq Y_j$ para todo $1 \leq j \leq K_2$
- Cada par de estações é ligado por no máximo uma ligação no total (ou seja, considerando ambos os sistemas)
- $1 \leq A, B \leq N$
- $A \neq B$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima:

- **Subtarefa 1 (31 pontos):** $K_2 = 0$, ou seja, todas as ligações são do sistema de metrô. (*Veja o exemplo 2.*)
- **Subtarefa 2 (32 pontos):** $N \leq 100$.
- **Subtarefa 3 (37 pontos):** Nenhuma restrição adicional.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (*elas não precisam ser resolvidas em ordem*). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
7 4 5 5 1 3 3 2 4 2 6 5 4 1 4 3 4 5 2 5 6 7 1 7	15

Explicação do exemplo 1: Este exemplo corresponde ao exemplo mostrado no enunciado.

Exemplo de entrada 2	Exemplo de saída 2
6 4 0 42 1 4 6 1 3 2 4 6 3 5	-1

Explicação do exemplo 2: A estação 5 não possui ligação com nenhuma outra estação, portanto é impossível chegar nela a partir da estação 3.

Oficina Mecânica

Autor: André Amaral de Sousa

Nome do arquivo: `mechanica.c`, `mechanica.cpp`, `mechanica.java`, `mechanica.js` ou `mechanica.py`

O dono de uma oficina mecânica te contratou para criar um software de gerenciamento. Com esse software, ele espera não só conseguir controlar os estoques dos produtos utilizados na oficina, mas também auxiliar no trabalho de seus funcionários e tornar a oficina mais eficiente.

Como um(a) excelente programador(a), você já implementou quase todo o software. Falta apenas uma funcionalidade: distribuir os carros a serem consertados entre os mecânicos de modo a garantir alta satisfação dos clientes. Após analisar os dados coletados na oficina sobre clientes passados, você descobriu que o fator mais importante para a satisfação de um cliente é seu *tempo de espera* para ser atendido, ou seja, o tempo em que ele fica esperando até o seu carro começar a ser consertado. Os clientes não se importam com o tempo de conserto de seus respectivos carros, pois eles sabem que os mecânicos fazem um excelente trabalho.

O tempo de conserto varia de acordo com o carro e com o mecânico: cada carro possui um *tempo base* T_i , já determinado durante o orçamento, que mede a quantidade de trabalho necessária para consertar o carro, e cada mecânico possui um *fator de trabalho* F_j , indicando quanto tempo ele demora para completar uma unidade de trabalho. Assim, o tempo necessário para o mecânico j consertar o carro i é $T_i \cdot F_j$. Vale ressaltar que cada mecânico só trabalha em um único carro por vez, e portanto precisa finalizar o conserto de um carro antes de começar a consertar o próximo.

Por exemplo, suponha que tenhamos quatro carros cujos tempos base são 5, 10, 15 e 20, e dois mecânicos cujos fatores de trabalho são 1 e 2. Uma distribuição possível é:

- O mecânico com fator 1 conserta os carros com tempos 20 e 5, nessa ordem.
- O mecânico com fator 2 conserta os carros com tempos 10 e 15, nessa ordem.

Essa distribuição possui os seguintes tempos de espera:

- O carro com tempo base 20 é atendido imediatamente, portanto seu tempo de espera é 0.
- O carro com tempo base 5 precisa esperar o carro com tempo base 20 ser consertado, portanto seu tempo de espera é $20 \cdot 1 = 20$.
- O carro com tempo base 10 é atendido imediatamente, portanto seu tempo de espera é 0.
- O carro com tempo base 15 precisa esperar o carro com tempo base 10 ser consertado, portanto seu tempo de espera é $10 \cdot 2 = 20$.

A soma total dos tempos de espera na distribuição acima é $0 + 20 + 0 + 20 = 40$. Porém, existe uma distribuição (que é ótima) onde a soma total dos tempos de espera é 20 – você consegue encontrá-la?

Dados o número N de carros esperando para serem atendidos, a quantidade M de mecânicos, o tempo base T_i de cada carro, e o fator de trabalho F_j de cada mecânico, determine a menor soma total dos tempos de espera dos carros, que corresponde à melhor forma de distribuir todos os carros entre os mecânicos.

Entrada

A primeira linha da entrada contém dois inteiros N e M representando o número de carros e o número de mecânicos, respectivamente.

A segunda linha contém N números inteiros. O i -ésimo desses números será T_i , representando o tempo base do i -ésimo carro.

A terceira e última linha da entrada contém M números inteiros. O j -ésimo desses números será F_j , representando o fator de trabalho do j -ésimo mecânico.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, a menor soma total dos tempos de espera em uma distribuição de todos os carros entre os mecânicos.

Restrições

- $1 \leq N \leq 100\,000$
- $1 \leq M \leq 100\,000$
- $1 \leq T_i \leq 10\,000$ para todo $1 \leq i \leq N$
- $1 \leq F_j \leq 10\,000$ para todo $1 \leq j \leq M$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima:

- **Subtarefa 1 (14 pontos):** $N \leq 1000$ e $M = 1$.
- **Subtarefa 2 (10 pontos):** $T_i = 1$ para todo $1 \leq i \leq N$ e $F_j = 1$ para todo $1 \leq j \leq M$.
- **Subtarefa 3 (16 pontos):** $F_j = 1$ para todo $1 \leq j \leq M$.
- **Subtarefa 4 (36 pontos):** $N \leq 1000$ e $M \leq 1000$.
- **Subtarefa 5 (24 pontos):** Nenhuma restrição adicional.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (*elas não precisam ser resolvidas em ordem*). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
4 2 5 10 15 20 1 2	20

Explicação do exemplo 1: Este exemplo corresponde ao exemplo mostrado no enunciado.

Exemplo de entrada 2	Exemplo de saída 2
4 1 1 1 1 1 3	18

Explicação do exemplo 2: Como há apenas um mecânico, ele consertará todos os carros. E como todos os tempos base são iguais, a ordem dos carros não afeta o resultado. Logo, podemos só consertar os carros em ordem. Os tempos de espera dos quatro carros são então, respectivamente, 0, 3, 6, e 9, para um total de $0 + 3 + 6 + 9 = 18$.

Exemplo de entrada 3	Exemplo de saída 3
4 1 15 5 10 20 3	150

Dominó Nlogônico

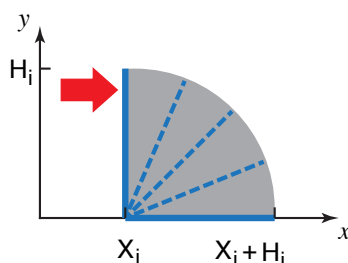
Autor: André Amaral de Sousa

Nome do arquivo: `domino.c`, `domino.cpp`, `domino.java`, `domino.js` ou `domino.py`

A Nlogônia é o lugar das coisas gigantes (por exemplo, o sistema de transporte de lá possui 100 000 estações!). Recentemente, seus habitantes têm se entretido com um novo jogo: *Dominó Nlogônico*. Como era de se esperar, o dominó da Nlogônia não é nada comum, e também pode ser gigante!

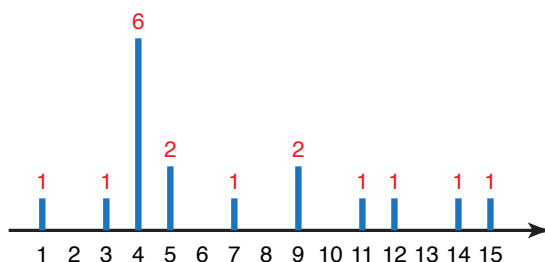
O jogo funciona da seguinte forma:

- N peças de dominó são posicionadas de pé, em linha reta.
- A i -ésima peça está em pé na posição X_i da reta e possui altura H_i .
- Quando uma peça é derrubada para a direita, ela cai e ocupa o intervalo de X_i a $X_i + H_i$ na reta, conforme mostrado na figura a seguir:

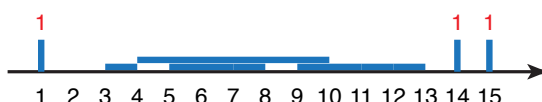


- Se, ao cair, a peça ocupar a posição de alguma outra peça (ou seja, se a posição da outra peça estiver no intervalo $[X_i, X_i + H_i]$), essa outra peça também cai, podendo derrubar mais peças, e assim sucessivamente.
- O objetivo do jogo é computar, para cada peça, quantos dominós iriam cair caso ela fosse derrubada para a direita.

Para entender este mecanismo, veja o exemplo abaixo:



A figura acima mostra 10 dominós. Caso o dominó na posição 3 fosse derrubado para a direita, o resultado final seria:



No total, 7 dominós iriam cair, pois:

- Ao ser derrubado para a direita, o dominó na posição 3 iria derrubar o dominó na posição 4.
- Ao ser derrubado para a direita, o dominó na posição 4 iria derrubar os dominós nas posições 5 e 9.
- Ao ser derrubado para a direita, o dominó na posição 5 iria derrubar o dominó na posição 7.
- Ao ser derrubado para a direita, o dominó na posição 9 iria derrubar o dominó na posição 11.
- Ao ser derrubado para a direita, o dominó na posição 11 iria derrubar o dominó na posição 12.

Observe que, toda vez que um dominó derrubado para a direita ocupa a posição de outro dominó, este outro dominó também cai, inclusive se a posição do outro dominó for um dos extremos do dominó derrubado (ou seja, as bordas de $[X_i, X_i + H_i]$).

Note também que o jogo pede a resposta para cada dominó de maneira independente: a resposta para um dado dominó é o número de dominós derrubados **assumindo que todos começam em pé em suas posições e apenas ele é derrubado inicialmente**, possivelmente gerando a queda de outros dominós. Assim, no exemplo acima, a resposta para o dominó na posição 9 é 3 (ao cair, ele derruba o dominó na posição 11, que por sua vez derruba o dominó na posição 12).

Já que você adora a Nlogônia, calcule a resposta para cada uma das peças e prove que você também consegue ganhar esse jogo!

Entrada

A primeira linha da entrada contém apenas um inteiro N , representando o número de peças de dominó.

A segunda linha contém N números inteiros. O i -ésimo desses números é X_i , representando a posição do dominó i na reta.

A terceira e última linha contém N números inteiros. O i -ésimo desses números é H_i , representando a altura do dominó i .

Saída

Seu programa deverá imprimir uma única linha contendo N números inteiros separados por espaço. O i -ésimo desses números deve ser a quantidade total de dominós que iriam cair caso o i -ésimo dominó (na ordem da entrada) fosse derrubado para a direita.

Restrições

- $1 \leq N \leq 300\,000$
- $1 \leq X_i \leq 1\,000\,000\,000$ para todo $1 \leq i \leq N$
- $1 \leq H_i \leq 1\,000\,000\,000$ para todo $1 \leq i \leq N$
- $X_i < X_{i+1}$ para todo $1 \leq i < N$, ou seja, os dominós são dados em ordem crescente de posição, e não existem dois dominós na mesma posição

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima:

- **Subtarefa 1 (18 pontos):** $N \leq 1\,000$.
- **Subtarefa 2 (22 pontos):** $H_i \leq H_{i+1}$ para todo $1 \leq i < N$. (Veja o exemplo 2.)
- **Subtarefa 3 (33 pontos):**
 - $X_i \leq 1\,000\,000$ para todo $1 \leq i \leq N$
 - $H_i \leq 100$ para todo $1 \leq i \leq N$
- **Subtarefa 4 (27 pontos):** Nenhuma restrição adicional.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (*elas não precisam ser resolvidas em ordem*). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
10 1 3 4 5 7 9 11 12 14 15 1 1 6 2 1 2 1 1 1 1	1 7 6 2 1 3 2 1 2 1

Explicação do exemplo 1: Este exemplo corresponde ao exemplo mostrado no enunciado.

Exemplo de entrada 2	Exemplo de saída 2
4 2 9 15 25 5 7 10 13	1 3 2 1

Pirâmide

Autor: Mateus Bezrutchka

Nome do arquivo: `piramide.c`, `piramide.cpp`, `piramide.java`, `piramide.js` ou `piramide.py`

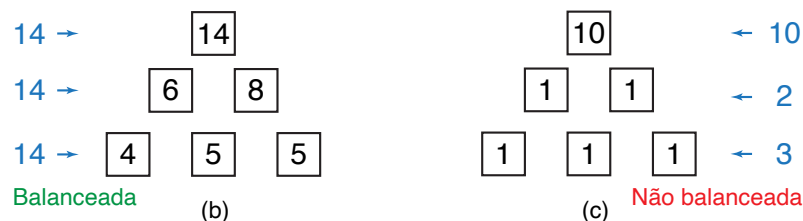
O mais novo lançamento da OBI (*Organização de Brincadeiras Infantis*) é o jogo *Balanceamento de Pirâmide*, que visa explorar as habilidades matemáticas das crianças. O jogo contém seis cubos, todos de mesmo tamanho mas com pesos possivelmente diferentes (os cubos são preenchidos com diferentes volumes de areia). O peso de cada cubo em gramas está escrito em uma de suas faces.



(a)

Em *Balanceamento de Pirâmide*, o objetivo é empilhar os seis cubos de modo a formar uma pirâmide de três andares (com três cubos no primeiro andar, dois no segundo e um no terceiro) tal que os pesos totais de cada andar (ou seja, a soma dos pesos dos cubos no andar) possuam o mesmo valor; nesse caso, dizemos que a pirâmide é *balanceada*. Por exemplo, se os cubos tem pesos (em gramas) 6, 5, 4, 14, 5 e 8, podemos obter uma pirâmide balanceada (com peso total 14 em cada andar) colocando os dois cubos de peso 5 e o cubo de peso 4 no primeiro andar, os cubos de pesos 6 e 8 no segundo andar, e o cubo de peso 14 no terceiro andar (veja a figura b).

Em toda unidade de *Balanceamento de Pirâmide*, deveria ser possível formar uma pirâmide balanceada com os seis cubos dados. Porém, devido a problemas na linha de produção, isso nem sempre é verdade. Recentemente, foi encontrado um exemplar contendo um cubo de peso 10 e cinco cubos de peso 1! Observe que não podemos obter uma pirâmide balanceada com esses cubos (só o cubo de peso 10 é mais pesado do que todos os outros juntos).



Você foi contratado como parte do controle de qualidade da OBI, e sua primeira missão é avaliar se uma unidade do jogo foi produzida corretamente: dados os pesos dos seis cubos, determine se é possível formar uma pirâmide balanceada com os cubos.

Entrada

A única linha da entrada contém seis números inteiros positivos X_i ($1 \leq i \leq 6$), indicando os pesos em gramas de cada um dos seis cubos, separados por espaços.

Saída

Seu programa deverá imprimir uma única linha contendo um único caractere: 'S' (sem aspas), se é possível formar uma pirâmide balanceada com os seis cubos, ou 'N' (sem aspas), se é impossível.

Restrições

- $1 \leq X_i \leq 1\,000$ para $1 \leq i \leq 6$

Informações sobre a pontuação

A tarefa vale 100 pontos.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
6 5 4 14 5 8	S

Explicação do exemplo 1: Esse exemplo é representado pela figura (b).

Exemplo de entrada 2	Exemplo de saída 2
10 1 1 1 1 1	N

Explicação do exemplo 2: Esse exemplo é representado pela figura (c).

Exemplo de entrada 3	Exemplo de saída 3
10 30 50 70 80 120	S

Explicação do exemplo 3: Podemos colocar os cubos de pesos 10, 30 e 80 no primeiro andar, os cubos de pesos 50 e 70 no segundo andar, e o cubo de peso 120 no terceiro andar, obtendo peso total de 120 por andar.

Exemplo de entrada 4	Exemplo de saída 4
4 8 12 24 20 16	N

Transportes

Autores: André Amaral de Sousa e Rafael Nascimento Soares

Nome do arquivo: `transportes.c`, `transportes.cpp`, `transportes.java`, `transportes.js` ou `transportes.py`

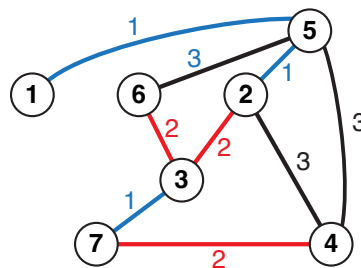
Ênia, a atual rainha da Nlogônia, é fascinada por meios de transporte de todos os tipos. Como consequência, a Nlogônia possui a melhor rede de transporte público do mundo inteiro.

Existem N estações espalhadas por todo o reino, K sistemas de transporte (metrô, ônibus, avião, barco, etc) e M ligações diretas entre pares de estações. Cada ligação pertence a um único sistema de transporte entre os K existentes, e pode ser percorrida nas duas direções. Para cada par de estações, existe no máximo uma ligação no total (ou seja, entre todos os sistemas).

Para usar os sistemas de transporte, o usuário precisa comprar passagens. Ênia estabeleceu as seguintes regras para estimular o transporte público:

- Cada um dos K sistemas de transporte tem um preço fixo de passagem. Assim, o preço de uma passagem do sistema i é P_i , independente da estação.
- A passagem só deve ser paga quando o usuário entrar no sistema de transporte.
- O usuário só pode entrar em um sistema por vez (por exemplo, para entrar no sistema de avião ele precisa sair do sistema de metrô).
- Após pagar o preço P_i para entrar no sistema i , o usuário pode se locomover o quanto desejar usando ligações do sistema i , contanto que ele não saia desse sistema. Caso o usuário troque de sistema, ele deverá pagar por uma nova passagem de preço P_i para retornar ao sistema i .

Por exemplo, considere a seguinte rede de transportes:



Perceba que há 7 estações e 9 ligações, das quais 3 são do sistema 1, 3 são do sistema 2 e 3 são do sistema 3. Suponha que os preços das passagens sejam $P_1 = 5$, $P_2 = 1$ e $P_3 = 7$ e que o usuário deseja fazer uma viagem da estação 1 para a estação 7. Temos várias opções de trajeto, como por exemplo (as setas indicam a ordem em que visitamos as estações):

- **Viagem 1:** $1 \rightarrow 5 \rightarrow 6 \rightarrow 3 \rightarrow 7$
 - A ligação entre as estações 1 e 5 é do tipo 1, logo precisamos pagar $P_1 = 5$ para entrar no sistema 1 pela primeira vez.
 - $5 \rightarrow 6$ usa o sistema 3, então precisamos pagar $P_3 = 7$ para entrar no sistema 3 (saindo do sistema 1).
 - $6 \rightarrow 3$ usa o sistema 2, então pagamos $P_2 = 1$.

- $3 \rightarrow 7$ usa o sistema 1, então pagamos $P_1 = 5$. Note que precisamos pagar novamente pela passagem do sistema 1 pois estamos reentrando nele.
- No total, a viagem custa $5 + 7 + 1 + 5 = 18$.
- **Viagem 2:** $1 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow 7$
 - $1 \rightarrow 5$ usa o sistema 1, então pagamos $P_1 = 5$.
 - $5 \rightarrow 2$ também usa o sistema 1. Como já entramos nesse sistema, não precisamos pagar novamente.
 - $2 \rightarrow 4$ usa o sistema 3, então trocamos de sistema e pagamos $P_3 = 7$.
 - $4 \rightarrow 7$ usa o sistema 2, então pagamos mais $P_2 = 1$.
 - No total, essa viagem custará $5 + 7 + 1 = 13$.
- **Viagem 3:** $1 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 7$
 - $1 \rightarrow 5$ usa o sistema 1, então pagamos $P_1 = 5$.
 - $5 \rightarrow 2$ também usa o sistema 1, então não pagamos nada.
 - $2 \rightarrow 3$ usa o sistema 2, então pagamos $P_2 = 1$ para trocar de sistema.
 - $3 \rightarrow 7$ usa o sistema 1, então pagamos $P_1 = 5$ para voltar ao sistema 1.
 - No total, essa viagem custará $5 + 1 + 5 = 11$.

Existem outras viagens que começam na estação 1 e terminam na estação 7. Porém, o menor custo total entre todas essas viagens é 11 (correspondente à viagem 3).

Ênia deseja saber o menor custo total para ir da estação A para a estação B , e para isso te contratou. Responda corretamente, ou ela te expulsará da Nlogônia!

Entrada

A primeira linha da entrada contém três inteiros N , M e K , representando o número de estações, o número de ligações entre as estações, e a quantidade de sistemas de transporte, respectivamente.

A segunda linha contém K números inteiros. O i -ésimo desses números é P_i , representando o preço da passagem do sistema de transporte i .

As próximas M linhas contém a descrição das ligações. A j -ésima dessas linhas tem três números V_j , U_j e T_j , representando que há uma ligação entre as estações V_j e U_j usando o sistema T_j .

A última linha da entrada contém dois números inteiros, A e B , representando a estação inicial e a estação final da viagem requisitada pela rainha.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o menor custo total de uma viagem que começa na estação A e termina na estação B .

Se não for possível fazer uma viagem de A a B usando a rede de transportes, imprima -1 .

Restrições

- $2 \leq N \leq 100\,000$
- $1 \leq M \leq 100\,000$
- $1 \leq K \leq 100\,000$

- $1 \leq P_i \leq 1\,000$ para todo $1 \leq i \leq K$
- $1 \leq V_j, U_j \leq N$ para todo $1 \leq j \leq M$
- $V_j \neq U_j$ para todo $1 \leq j \leq M$
- $1 \leq T_j \leq K$ para todo $1 \leq j \leq M$
- Cada par de estações é ligado por no máximo uma ligação no total (ou seja, os pares de estações V_j, U_j são todos distintos)
- $1 \leq A, B \leq N$
- $A \neq B$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima:

- **Subtarefa 1 (11 pontos):** $K = 1$. (Veja o exemplo 2.)
- **Subtarefa 2 (16 pontos):** $K = 2$, $N \leq 100$ e $P_1 = P_2 = 1$. (Veja o exemplo 3.)
- **Subtarefa 3 (34 pontos):** $K \leq 10$, $N \leq 10\,000$.
- **Subtarefa 4 (18 pontos):** $K \leq 100$.
- **Subtarefa 5 (21 pontos):** Nenhuma restrição adicional.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (*elas não precisam ser resolvidas em ordem*). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
7 9 3 5 1 7 1 5 1 5 2 1 7 3 1 3 6 2 2 3 2 7 4 2 2 4 3 5 6 3 5 4 3 1 7	11

Explicação do exemplo 1: Este exemplo corresponde ao exemplo mostrado no enunciado.

Exemplo de entrada 2 6 4 1 50 1 4 1 6 1 1 3 2 1 4 6 1 3 5	Exemplo de saída 2 -1
Exemplo de entrada 3 5 5 2 1 1 1 2 1 5 2 1 1 3 2 4 3 1 4 5 2 2 3	Exemplo de saída 3 2

Trio de Bonecas

Autor: André Amaral de Sousa

Nome do arquivo: `bonecas.c`, `bonecas.cpp`, `bonecas.java`, `bonecas.js` ou `bonecas.py`

Alice é a gerente de produção de uma fábrica de bonecas chinesas. Nessa fábrica, todas as bonecas possuem o mesmo formato, variando apenas de tamanho. As bonecas são vendidas sempre em trios.



Para um trio formado por bonecas de tamanhos A , B e C , com $A \leq B \leq C$, o *balanceamento* do trio é definido como $(A - B)^2$. De acordo com o controle de qualidade da fábrica, idealmente cada trio deveria ter balanceamento 0, ou seja, conter duas bonecas de mesmo tamanho e uma terceira boneca de tamanho maior ou igual a essas duas; porém, isso nem sempre é possível devido a limitações de estoque.

Alice acabou de receber um pedido urgente: o consulado Chinês fará um grande evento no qual pretende distribuir K trios de bonecas, e precisa que Alice envie imediatamente todos os trios. Existem na fábrica N bonecas de diversos tamanhos, ainda não separadas em trios, e Alice não sabe se conseguirá formar K trios ideais.

Devido à urgência do evento, Alice e o consulado fizeram um acordo: o consulado aceitará trios que não sejam ideais; porém, para garantir um bom evento, Alice deve enviar uma caixa com K trios de bonecas cuja soma dos balanceamentos seja a menor possível.

Por exemplo, suponha que Alice possua 7 bonecas com tamanhos 11, 1, 7, 5, 16, 8 e 15, e que o consulado tenha solicitado 2 trios de bonecas. Algumas configurações possíveis para envio são:

- **Configuração 1:** trios (7, 8, 15) e (1, 5, 16).
 - O primeiro trio possui balanceamento $(8 - 7)^2 = 1$.
 - O segundo trio possui balanceamento $(5 - 1)^2 = 16$.
 - O balanceamento total dessa configuração é $1 + 16 = 17$.
- **Configuração 2:** trios (5, 7, 16) e (8, 11, 15).
 - O primeiro trio possui balanceamento $(7 - 5)^2 = 4$.
 - O segundo trio possui balanceamento $(11 - 8)^2 = 9$.
 - O balanceamento total dessa configuração é $4 + 9 = 13$.

Existem outras configurações possíveis, mas a menor soma de balanceamentos possível é 13 (correspondente à configuração 2).

Ajude Alice a calcular a soma de balanceamentos mínima em um conjunto com K trios de bonecas.

Entrada

A primeira linha de entrada contém dois inteiros N e K , representando, respectivamente, o número de bonecas e a quantidade de trios solicitados pelo consulado Chinês.

A segunda linha contém N números inteiros. O i -ésimo desses números será T_i , representando o tamanho da i -ésima boneca.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, a menor soma de balanceamentos possível em uma caixa com K trios de bonecas.

Restrições

- $3 \leq N \leq 10\,000$
- $1 \leq K \leq 3\,000$
- $3K \leq N$
- $1 \leq T_i \leq 100\,000$ para todo $1 \leq i \leq N$

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima:

- **Subtarefa 1 (13 pontos):** $1 \leq T_i \leq 2$ para todo $1 \leq i \leq N$.
- **Subtarefa 2 (28 pontos):** $N \leq 30$.
- **Subtarefa 3 (33 pontos):** $N \leq 1\,000$ e $K \leq 100$.
- **Subtarefa 4 (26 pontos):** Nenhuma restrição adicional.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (*elas não precisam ser resolvidas em ordem*). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
7 2 11 1 7 5 16 8 15	13

Explicação do exemplo 1: Este exemplo corresponde ao exemplo mostrado no enunciado.

Exemplo de entrada 2	Exemplo de saída 2
8 2 2 1 2 2 2 1 2 2	0

Fast-Food

Autor: Mateus Bezrutchka

Nome do arquivo: `fastfood.c`, `fastfood.cpp`, `fastfood.java`, `fastfood.js` ou `fastfood.py`

Após a descoberta de um tesouro escondido terminar uma maldição milenar, a histórica ilha da Quadradônia está pronta para ser repovoada. A expectativa mundial é de que a ilha se torne um grande ponto turístico, recebendo visitantes de todo o planeta. Como você pode imaginar, a Quadradônia tem esse nome porque as suas ruas formam um grid quadriculado e todos os prédios comerciais estão localizados em intersecções entre duas ruas.

Você foi contratado como programador na *Fast-Food Turbo (FFT)*, uma rede de franquias de restaurantes de comida rápida. De olho nas oportunidades abertas na Quadradônia, a FFT adquiriu N prédios comerciais na ilha, e deseja abrir, nesses prédios, unidades de duas das suas redes mais populares: o restaurante de hambúrgueres quadrados Quaburguer e o restaurante de pizzas quadradas Pizzatro.

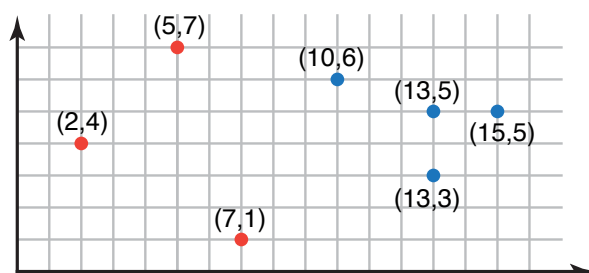
Devido à localização remota da ilha, é muito difícil levar mercadorias para a Quadradônia. Por isso, quando faltam ingredientes em um restaurante da ilha, a solução mais rápida sempre é pedir esses ingredientes a algum outro restaurante da ilha. Se outro restaurante tiver os ingredientes necessários, ele manda um funcionário de carro para fazer a entrega ao restaurante necessitado. Porém, como o Quaburguer e a Pizzatro usam ingredientes bem diferentes, um restaurante só pode fornecer ingredientes para outro restaurante da mesma rede.

Como não existem engarrafamentos na Quadradônia, o tempo que um entregador leva para ir de um prédio a outro é sempre proporcional à *distância quadradônica* entre os prédios, definida como a distância que precisa ser percorrida em ruas da ilha para ir de um prédio a outro. Mais formalmente, se os prédios i e j têm coordenadas (X_i, Y_i) e (X_j, Y_j) , respectivamente, a distância quadradônica entre eles é definida como

$$d_{\text{quad}}(i, j) = |X_i - X_j| + |Y_i - Y_j|.$$

A FFT precisa decidir, para cada prédio, de qual rede o restaurante desse prédio fará parte. Como a empresa não consegue prever quais prédios precisarão de ingredientes, os membros do conselho decidiram que eles querem minimizar o pior caso possível, ou seja, a distância d_{quad} máxima entre todos os pares de restaurantes de uma mesma rede.

A figura abaixo ilustra um exemplo para $N = 7$. Observe que, se os prédios 1 e 4 pertencerem à mesma rede, a resposta será no mínimo a distância entre eles, $d_{\text{quad}}(1, 4) = |2 - 13| + |4 - 5| = 12$. Porém, podemos conseguir uma distância máxima igual a 8 se tivermos uma rede com os prédios (1, 3, 5) e a outra com os prédios (2, 4, 6, 8):



Existem outras configurações possíveis, mas 8 é a distância mínima para esse conjunto de prédios.

Sua tarefa é: dadas as coordenadas dos N prédios que a FFT adquiriu, distribua os N prédios entre as duas redes de modo a minimizar a distância quadradônica máxima entre dois restaurantes de uma mesma rede.

Entrada

A primeira linha da entrada contém um único inteiro N , o número de prédios da Quadradônia adquiridos pela FFT.

A segunda linha da entrada contém N inteiros. O i -ésimo desses inteiros, X_i , representa a primeira coordenada do i -ésimo prédio.

A terceira e última linha da entrada contém N inteiros. O i -ésimo desses inteiros, Y_i , representa a segunda coordenada do i -ésimo prédio.

Saída

Seu programa deverá produzir uma única linha contendo um único inteiro, a menor distância quadradônica máxima possível entre dois restaurantes de uma mesma rede.

Restrições

- $1 \leq N \leq 300\,000$
- $1 \leq X_i \leq 500\,000\,000$ para todo $1 \leq i \leq N$
- $1 \leq Y_i \leq 500\,000\,000$ para todo $1 \leq i \leq N$
- Observe que é **possível** que dois prédios tenham as mesmas coordenadas.

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima:

- **Subtarefa 1 (23 pontos):** $N \leq 16$.
- **Subtarefa 2 (35 pontos):** $N \leq 2\,000$.
- **Subtarefa 3 (42 pontos):** Nenhuma restrição adicional.

Seu programa pode resolver corretamente todas ou algumas das subtarefas acima (*elas não precisam ser resolvidas em ordem*). Sua pontuação final na tarefa é a soma dos pontos de todas as subtarefas resolvidas corretamente por qualquer uma das suas submissões.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
7 2 10 7 13 5 13 15 4 6 1 5 7 3 5	8

Explicação do exemplo 1: Esse exemplo corresponde ao exemplo do enunciado.